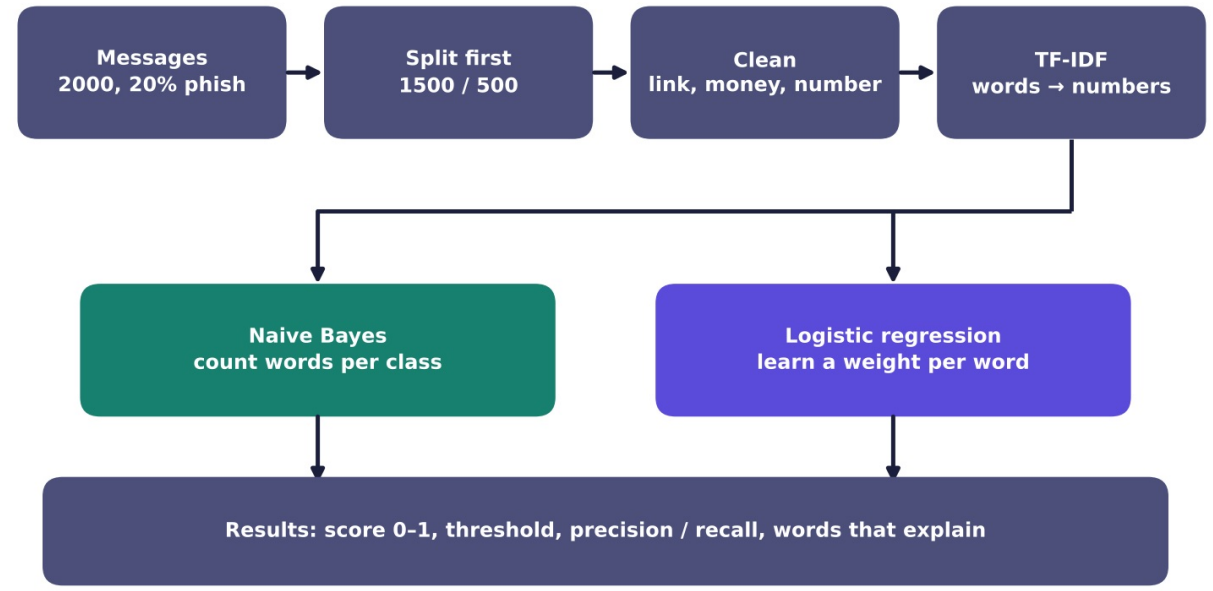
Lab 04: Phish or Legit

What you build: a web application that classifies a message as phishing or legitimate, returns the classifier's score, and explains the decision with per-feature contributions. **What you receive:** the code, the dataset (2,000 synthetic messages, prevalence of the positive class 0.20) and the evaluation script. You do not implement the model.



Workflow of the lab

1. What is the problem?

Phishing detection is a **binary text classification** task: given a message, predict whether it is phishing (positive class) or legitimate (negative class).

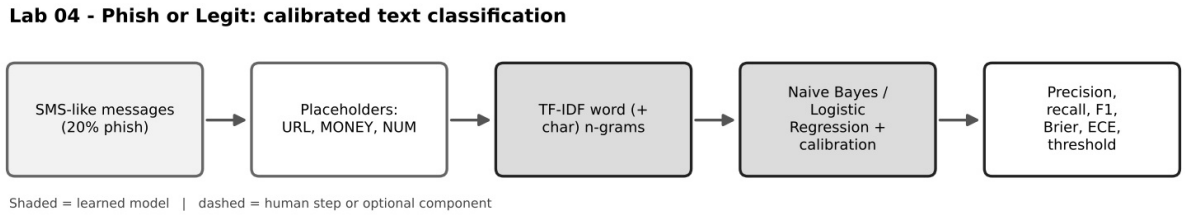
- **Class imbalance.** Only 20% of the messages are phishing. A majority-class classifier that always predicts "legit" reaches 0.80 accuracy and a recall of 0. Accuracy is therefore not an informative metric here.
- **Asymmetric error costs.** A false negative (missed phishing) and a false positive (a legitimate message flagged) do not cost the same, so the operating point must be chosen deliberately.
- **Calibration.** A classifier score is not necessarily a probability. Naive Bayes scores in particular tend to be over-confident.

2. What is the solution we apply?

We build a **supervised text classification pipeline** and compare two classifiers on the same features.

- **Features:** TF-IDF over word unigrams and bigrams (730 features), after replacing URLs, amounts and numbers by placeholder tokens.
- **Classifiers: Multinomial Naive Bayes** (generative, based on class-conditional word frequencies) and **logistic regression** (discriminative, one learned coefficient per feature, L2 regularisation).
- **Decision rule:** predict phishing when the score is above a **decision threshold**. The threshold is selected from a stated policy and is not fixed at 0.5.
- **Explainability:** for logistic regression, the contribution of each feature to the log-odds is its TF-IDF value multiplied by its coefficient.
- **Methodology:** a stratified train/test split performed before any fitting, with the vectoriser and the classifier fitted inside a single `Pipeline` on the training set only.

3. How do we apply it?



Pipeline

1. **Inspect the data.** Read 10 phishing and 10 legitimate messages and note the discriminative terms.
2. **Run the evaluation script.** It performs the stratified split (1,500 training / 500 test messages, seed 42), fits both pipelines on the training set and evaluates them on the test set.
3. **Read the metrics.** Fill in a table: classifier, precision, recall, F1, Brier score. At threshold 0.5 the expected values are about 0.75 precision and 0.71 recall for logistic regression, and 0.61 precision and 0.80 recall for Naive Bayes.
4. **Select a threshold.** Raising the threshold increases precision and decreases recall. Choose a value and justify it with a policy sentence, for example “we accept missed phishing rather than blocking legitimate messages”.
5. **Explain predictions.** For a chosen message, list the features with the largest positive and negative contributions to the log-odds.
6. **Build the application.** See the deliverables.
7. **Deploy.** Publish the application and test the public URL.

4. Deliverables (the most important part)

The work is assessed mainly on what you can demonstrate: a working application available at a public URL.

A. A web application

Build it with **Streamlit** (recommended, Python) or **React** (more work). It must provide:

1. A text input for a message and a button to classify it.
2. The predicted class and the classifier score.
3. A **decision threshold slider** that changes the predicted class live.
4. A **per-feature explanation**: the features that increased the score and the features that decreased it.
5. At least 4 selectable example messages (two phishing, two legitimate, including one that your classifier misclassifies).
6. A visible notice: “This tool supports a human decision and does not block messages by itself. The data is synthetic.”

Optional: a selector between the two classifiers, and a precision/recall versus threshold plot.

B. A public URL

- **Streamlit Community Cloud** (simplest): push the project to a GitHub repository with its requirements file, connect the repository on share.streamlit.io and select the application file.
- **Vercel** (for a React application): Vercel serves the front end but does not run the Python model. You need either a separate API that hosts the model, or the exported model coefficients scored in the browser. Choose this only if you are comfortable with it.
- Open the URL in a private browser window and test it before submitting.

C. A one-page report

- The problem and the solution in 5 lines.
- The results table (classifier, precision, recall, F1, Brier score).
- The selected threshold and the policy sentence that justifies it.
- Two misclassified messages, with an explanation based on the feature contributions.

D. What you hand in

1. The GitHub repository URL, with a short README (purpose, how to run).
2. The public application URL.
3. The report (PDF).
4. Two screenshots of the application.

Common mistakes

- Reporting accuracy only.
- Selecting the threshold on the test set and then reporting the test score as unbiased.
- Interpreting a Naive Bayes score as a calibrated probability.
- Submitting a URL that only works on your own machine.

5. Extensions

- Add character n-grams (`char_wb`, 2 to 5) to handle obfuscated tokens such as “paypa1”.
- Apply sigmoid (Platt) calibration with cross-validation and show a reliability diagram in the application.
- Accept a CSV of messages and export the flagged ones.
- Evaluate on the real UCI SMS Spam collection (note that it is a spam dataset, not a phishing dataset).
- Add a feedback button (“misclassified”) that logs messages for later retraining.

6. Limits and ethics

- The data is **synthetic**, with fictional brands. Real phishing evolves continuously, so performance on real traffic would be lower.
- Do not use real personal messages without consent, and remove phone numbers and names from any real data.
- The classifier supports a human decision. It must not silently block or delete messages.
- Report both error types honestly: false positives affect legitimate senders, false negatives affect victims.
- Once the discriminative features are known, an adversary can avoid them, so the model requires regular retraining.

7. Acronyms and definitions

Term	Definition
API	Application Programming Interface: a service that another application calls to get a result (here, to score a message)
Brier score	Mean squared error between the predicted probability and the true label (lower is better)
Calibration	Agreement between predicted probabilities and observed frequencies
Class imbalance	Unequal class sizes (here 20% phishing, 80% legitimate)
Data leakage	Information from the test set influencing training (vocabulary, idf values, coefficients)
Decision threshold	Score above which a message is classified as phishing
ECE	Expected Calibration Error: the average gap between predicted probability and observed frequency, computed per bin of scores
F1	Harmonic mean of precision and recall
FN	False negative: a phishing message classified as legitimate
FP	False positive: a legitimate message classified as phishing
L2	L2 regularisation: a penalty on the squared coefficients of logistic regression that limits overfitting
Log-odds	log(p / (1 - p)); logistic regression is linear in the log-odds, so each feature adds a contribution to it
Precision	TP / (TP + FP): the share of flagged messages that are phishing
Recall	TP / (TP + FN): the share of phishing messages that are flagged
TF-IDF	Term Frequency - Inverse Document Frequency: a feature weight that is high for terms frequent in a message and rare across messages
TP	True positive: a phishing message classified as phishing
URL	Uniform Resource Locator: a web address